Artificial World: Un Motor de Simulación de Civilizaciones Emergentes con Agentes Autónomos Basados en Utilidad

Autor: Cosigein SL **Fecha:** 2026-03-08 **Repositorio:** `artificial-word` **Versión:** Sesión fundacional, semilla 42, 200 ticks

Resumen

*Presentamos **Artificial World**, un motor de simulación de vida artificial 2D orientado a la generación de civilizaciones emergentes con memoria, héroes, refugios y comunidades. El sistema implementa agentes autónomos cuya toma de decisiones se fundamenta en una función de utilidad multidimensional que integra estado fisiológico, rasgos de personalidad, memoria espacial y social, relaciones inter-agente, directivas externas y reglas de autonomía de supervivencia. Describimos la arquitectura del motor de decisión, la estructura del entorno simulado, el protocolo de sesión fundacional reproducible y los resultados observados en una sesión canónica de 200 ticks con 8 entidades sobre un mapa de* 60×60 *celdas. Los resultados muestran un estado de tensión sostenida con supervivencia total de la población pero con hambre crítica en el 87.5 % de los agentes al final del experimento, evidenciando la presión evolutiva del entorno y la insuficiencia del mecanismo de búsqueda de recursos en condiciones de escasez prolongada. El sistema es reproducible mediante semilla fija y está disponible como motor Python open source con persistencia SQLite.*

Palabras clave: *vida artificial, agentes autónomos, función de utilidad, civilizaciones emergentes, simulación 2D, comportamiento emergente, memoria social, modo sombra.*

1. Introducción

La simulación de sistemas de vida artificial ha sido un campo fértil desde los trabajos fundacionales de Conway (1970) con el Juego de la Vida [2] y los ecosistemas de Reynolds (1987) con Boids [5]. Sin embargo, la mayoría de los sistemas clásicos priorizan la simplicidad de las reglas locales sobre la riqueza del estado interno de los agentes.

Artificial World parte de una premisa distinta: **los agentes deben tener estado interno complejo, memoria persistente y capacidad de decidir mediante ponderación de utilidades**, de forma que el comportamiento colectivo emergente sea consecuencia de decisiones individualmente racionales bajo restricción.

El objetivo del sistema no es la optimización de una métrica global, sino la generación de **historia emergente**: secuencias de eventos que den lugar a crónicas, memorias y finalmente a civilizaciones con identidad propia. La unidad fundamental no es el agente individual sino el **refugio** — un nodo de supervivencia colectiva alrededor del cual se organiza la comunidad.

Este trabajo documenta: (1) arquitectura del motor de decisión por utilidad; (2) entorno simulado y parámetros; (3) protocolo de sesión fundacional reproducible; (4) resultados de la sesión canónica (semilla 42, 200 ticks); (5) análisis de comportamiento emergente; (6) limitaciones y trabajo futuro.

2. Trabajo Relacionado

2.1. Vida Artificial y Agentes Autónomos

Los sistemas de vida artificial basados en agentes (Langton, 1989 [3]) se dividen en dos familias: **sistemas de reglas locales simples** (autómatas celulares, Boids) y **sistemas de agentes con estado interno** (Creatures, The Sims, Dwarf Fortress [1]). Artificial World pertenece a la segunda familia, con énfasis en la **arquitectura de decisión por utilidad** y en la **persistencia de memoria individual**.

2.2. Utilidad en Agentes Deliberativos

La teoría de la utilidad esperada [7] ha sido adaptada extensamente para sistemas multiagente. La arquitectura se inspira en los sistemas **BDI** (Rao & Georgeff, 1991 [4]), pero reemplaza la selección de intenciones por una función de puntuación continua que permite graduación natural entre opciones.

2.3. Sistemas de Refugio y Comunidad

La noción de refugio como unidad de organización social tiene antecedentes en Dwarf Fortress, RimWorld [6] y Caves of Qud. Artificial World formaliza el refugio como **entidad de primer orden** con estado propio, en lugar de tratarlo como mero artefacto del mapa.

3. Arquitectura del Sistema

3.1. Visión General

El sistema se compone de las capas mostradas en el Listado 1. Dispone de dos motores independientes:

- **Motor Python** (Python 3.11+ / pygame): simulación canónica, alta fidelidad; persistencia SQLite (`mundo_artificial.db`).
- **Motor JS** (Node.js / Express): demo web, HeroRefuge, API; persistencia SQLite (`audit_simulacion.db`).

La simulación avanza por ticks discretos. En cada tick, cada entidad ejecuta el ciclo: *Percepción → Generación de candidatos → Puntuación → Selección.*

Listing 1: Árbol de módulos principal

```
principal.py
|-- nucleo/simulacion.py
|    |-- agentes/motor_decision.py
|    |-- acciones/          (13 tipos)
|    |-- sistemas/          (Watchdog,
    Modo Sombra)
|    `-- mundo/mapa.py      (grid 60x60)
|-- sistemas/cronica_fundacional.py
|-- backend/src/            (API Node.js)
`-- frontend/src/           (React +
    Tailwind)
```

3.2. Función de Utilidad

La puntuación de una acción a para el agente i en el tick t es:

$$U(a,i,t) = U_{\text{base}}(a) + \sum_k \delta_k(a,i,t) + \varepsilon \qquad (1)$$

donde $U_{\text{base}}(a)$ es la utilidad intrínseca de la acción (Tabla 1), δ_k son modificadores contextuales (Tabla 2), y $\varepsilon \sim \mathcal{U}(-0{,}08, 0{,}08)$ es ruido determinista por semilla.

Cuadro 1: Utilidades base por acción

Acción	U_{base}
`comer`	0.65
`recoger_comida`	0.55
`explorar`	0.50
`huir`	0.40
`descansar`	0.30
`mover`	0.30
`recoger_material`	0.30
`ir_refugio`	0.25
`evitar`	0.25
`compartir`	0.20
`robar`	0.20
`seguir`	0.20
`construir`	0.18

La capa de autonomía actúa como **override de emergencia**: si hambre $> 0{,}85$, se penaliza exploración; si energía $< 0{,}15$, se penaliza movimiento y robo; si riesgo $> 0{,}8$, se potencia la huida sobre cualquier directiva.

3.3. Catálogo de Acciones

El sistema implementa 13 tipos de acción, cada uno con condición de viabilidad. Si ninguna es viable, el sistema garantiza fallback a `DESCANSAR`.

Cuadro 2: Modificadores de la función de utilidad

Modificador	Efecto
hambre	↑ comer/recoger cuando hambre > umbral
energía	↑ descansar cuando energía < 0,6
riesgo	↑ huir/evitar cuando riesgo > 0,3
rasgo	Modifica preferencias por arquetipo
oscilación	−0,40 sobre retorno a celda anterior
memoria	Bonus hacia comida recordada
relaciones	Confianza/hostilidad/miedo con cercanos
directivas	Órdenes externas con prioridad alta
autonomía	Override supervivencia sobre directivas

3.4. Rasgos de Agente

Los agentes sociales tienen 5 arquetipos: *Cooperativo*, *Neutral*, *Agresivo*, *Explorador* y *Oportunista*. Los agentes de tipo `gato` tienen su propia tabla de 5 rasgos: *Curioso*, *Apegado*, *Independiente*, *Territorial* y *Oportunista*.

3.5. Memoria de Agente

Cada agente mantiene dos registros:

- **Memoria espacial**: posiciones de recursos observados recientemente; alimenta directamente el modificador *memoria* de la ec. (1).
- **Memoria social**: relaciones con otros agentes en tres dimensiones: `confianza`, `hostilidad`, `miedo`.

3.6. Modo Sombra y Watchdog

El **Modo Sombra** es una simulación especular paralela que permite evaluar contrafácticos sin alterar el mundo observable. El **Watchdog** monitoriza condiciones anómalas y emite alertas estructuradas con nivel (`WARN`, `CRITICAL`), código semántico, tick de emisión y entidad afectada.

4. Entorno Experimental

4.1. Parámetros de la Sesión Canónica

Cuadro 3: Parámetros de la sesión fundacional

Parámetro	Valor
Semilla aleatoria	42
Dimensiones del mapa	60 × 60 celdas
Entidades iniciales	8
Ticks ejecutados	200
Fundador	Tryndamere
Refugio fundador	Refugio Fundador
Semilla de civilización	default
Timestamp	2026-03-08T12:53:48

4.2. Composición de la Población

Cuadro 4: Estado final de las 8 entidades (tick 200)

Nombre	Tipo	H	E
Ana	social	0.68	0.38
Bruno	social	1.00	0.36
Clara	social	1.00	0.26
David	social	0.90	0.18
Eva	social	1.00	0.40
Félix	social	1.00	0.38
Amiguisimo	gato	1.00	0.30
Tryndamere	social	1.00	0.40

H = hambre [0=saciado, 1=crítico]; E = energía [0=agotado, 1=pleno

La población incluye 7 entidades de tipo `social` y 1 de tipo `gato` (Amiguisimo). Todas mantienen salud = 1,0 al final, indicando que ninguna alcanzó el estado de muerte.

5. Resultados

5.1. Métricas de Acción Agregadas

La dominancia de `SEGUIR` (62.4 %) revela un patrón emergente: ante la escasez de recursos, los

Cuadro 5: Distribución de acciones en 200 ticks

Acción	Ejec.	Proporción
SEGUIR	998	62.4 %
DESCANSAR	492	30.8 %
RECOGER_RECURSO	55	3.4 %
COMER	55	3.4 %
Total	**1.600**	**100 %**

agentes con rasgo social priorizan la cohesión grupal sobre la búsqueda activa individual. Este resultado es consistente con la hipótesis de que el modificador de relaciones sociales supera al modificador de hambre en los estadios medios del experimento.

5.2. Estado Final de la Población

- **Veredicto:** TENSIÓN
- **Entidades vivas:** 8/8 (supervivencia total)
- **Hambre máxima:** 1.00 (crítica)
- **Hambre promedio:** 0.9475
- **Energía mínima:** 0.18
- **Energía promedio:** 0.3325
- **Alertas emitidas:** 65

5.3. Análisis de Alertas Watchdog

Las 65 alertas se distribuyen en dos códigos semánticos: `HAMBRE_SIN_COMIDA_DISPONIBLE` (WARN) y `HAMBRE_CRITICA_SIN_RESPUESTA` (CRITICAL). Se concentran en los ticks 189 y 199, indicando presión crítica sostenida en la fase final (ticks 180–200). Los agentes afectados (Tryndamere, Amiguisimo, Eva, Clara, Bruno, Félix) están ubicados en la mitad izquierda del mapa ($x < 15$), zona con menor densidad de recursos en esta configuración de semilla.

5.4. Dispersión Espacial

Los vectores de posición final revelan fragmentación en tres clusters:

- **Cluster NW** ($x < 15$, $y > 40$): Tryndamere (2,51), Félix (0,52), Amiguisimo (12,42).
- **Cluster SW** ($x < 30$, $y < 25$): David (10,22), Eva (13,5), Clara (25,5).
- **Posición aislada**: Bruno (0,34), Ana (47,21).

6. Discusión

6.1. Emergencia del Comportamiento Social

El resultado más significativo es la dominancia de `SEGUIR` (62.4 %) en un contexto de escasez severa. Este comportamiento no fue codificado como respuesta a la escasez, sino que emerge de la interacción entre: (1) el modificador de relaciones (confianza alta → bonus `SEGUIR`); (2) la penalización de autonomía sobre exploración cuando hambre $> 0{,}85$; y (3) la ausencia de comida que hace inviables `COMER` y `RECOGER` la mayor parte del tiempo.

Esto produce una **trampa dinámica**: los agentes cohesionados mantienen relaciones de alta confianza que refuerzan el seguimiento mutuo, pero esto reduce la exploración necesaria para encontrar recursos.

6.2. El Problema del Agotamiento Zonal

La concentración de alertas en el cluster oeste sugiere que la semilla 42 genera un mapa con gradiente de recursos que desfavorece las zonas de borde oeste. Los agentes allí ubicados sufren hambre crítica crónica por **agotamiento zonal** — han consumido los recursos locales y el modificador anti-oscilación ($-0{,}40$) dificulta el retroceso eficiente.

6.3. El Caso Anómalo: Ana

Ana es la única entidad con hambre sub-crítica ($H = 0{,}68$), ubicada en $(47, 21)$ — la posición más alejada del grupo. Su aislamiento espacial, no intencional sino resultado de la exploración individual, resultó adaptativo: sin competencia zo-

nal, tuvo acceso exclusivo a recursos del cuadrante NE.

Este resultado sugiere que, bajo presión de escasez, la **divergencia individual de la cohesión grupal** puede ser una estrategia emergentemente superior, aunque el sistema no la codifica como tal.

6.4. Reproducibilidad

La sesión es completamente reproducible mediante semilla fija:

Listing 2: Comando de reproducción

```
python cronica_fundacional.py \
  --seed 42 --ticks 200 \
  --founder "Tryndamere" \
  --refuge "Refugio␣Fundador"
```

7. Modelo de Civilizaciones Vivas

7.1. Entidades de Dominio

Cuadro 6: Entidades del modelo de dominio

Entidad	Rol
`World`	Contenedor global
`CivilizationSeed`	Semilla tipo fundacional
`Refuge`	Unidad base de supervivencia
`Hero`	Agente histórico
`Community`	Agrupación viva
`MemoryEntry`	Registro de memoria
`HistoricalRecord`	Evento histórico
`Territory`	Lectura estratégica 2D

7.2. Flujo Fundador

El flujo mínimo viable es: *Semilla* → *Refugio* → *Civilización naciente*. El usuario elige una `CivilizationSeed`, nombra al héroe fundador y al refugio; el sistema crea entidades, comunidad y crónica fundacional.

7.3. Estrategia 2D / 3D

- **2D** (implementada): verdad sistémica — mapa, grid, rutas, nodos, recursos, refugios, influencia, fronteras.
- **3D** (roadmap): encarnación futura — héroes de gran presencia, refugios encarnados, eventos históricos visuales.

8. Verificación y Tests

El sistema cuenta con 11 suites de tests Python y 45 tests en backend Node/Vitest, con un total verificado de **68+ tests** pasando en CI en cada push. Las suites cubren: ciclo tick, acciones, decisión, crónica fundacional, Modo Sombra, combate, interacciones sociales, Watchdog e integración producción completa.

Para ejecutar:

Listing 3: Runner de tests

```
$env:SDL_VIDEODRIVER="dummy"
$env:SDL_AUDIODRIVER="dummy"
python pruebas/run_tests_produccion.py
```

9. Limitaciones y Trabajo Futuro

9.1. Limitaciones Actuales

1. **Ausencia de integración Python/JS**: los dos motores son independientes sin comunicación en tiempo real.
2. **Escasez de hitos narrativos**: solo 2 hitos en 200 ticks (inicio y cierre); eventos intermedios no se registraron en esta sesión.
3. **Comunidad no simulada**: `Community` existe como contrato de datos, pero no como simulación social rica.
4. **Diplomacia y linajes**: civilizaciones completas con guerras y migraciones permanecen en roadmap.
5. **3D no implementada**: la capa de encarnación visual no existe aún como runtime.

6. **Capturas visuales**: la crónica aún no incluye screenshots automáticos de ticks clave.

9.2. Trabajo Futuro

Las próximas líneas de desarrollo son: enriquecimiento de `Refuge` y `Community` con normas sociales emergentes; historia profunda con más eventos alimentando `MemoryEntry`; mecanismo de migración grupal ante agotamiento zonal; implementación completa de las 7 semillas de civilización (tribu, tecnócrata, espiritual, guerrero, comerciante, paranoica, decadente); e integración Python/JS con puente en tiempo real.

10. Conclusiones

Artificial World demuestra que un motor de simulación relativamente compacto (arquitectura Python 2D, 13 tipos de acción, función de utilidad con 9 modificadores) es capaz de producir comportamientos emergentes no triviales y verificables. La sesión canónica (semilla 42, 200 ticks) revela:

- **Supervivencia total** bajo presión de escasez severa.
- **Emergencia de cohesión social** como respuesta a la escasez, con el efecto paradójico de reducir la exploración individual.
- **Divergencia individual exitosa** como estrategia no codificada (caso Ana).
- **Agotamiento zonal** como causa primaria de hambre crítica, no fallo algorítmico.
- **Reproducibilidad completa** mediante semilla fija y protocolo documentado.

El sistema establece las bases para un motor creador de mundos compacto y reutilizable. La tesis de producto — **civilizaciones vivas con refugios, héroes y memoria** — es técnicamente defendible en su estado actual en la dimensión 2D, con la capa de encarnación 3D explícitamente marcada como roadmap.

"Empieza con un refugio. Elige una semilla.
Mira nacer tu civilización."

Referencias

[1] Adams, T. (2006). *Dwarf Fortress*. Bay 12 Games.

[2] Conway, J. H. (1970). The Game of Life. *Scientific American*, 223(4), 4–10.

[3] Langton, C. G. (1989). Artificial Life. *Artificial Life*, SFI Studies in the Sciences of Complexity, 1–47.

[4] Rao, A. S., & Georgeff, M. P. (1991). Modeling Rational Agents within a BDI-Architecture. *KR '91*, 473–484.

[5] Reynolds, C. W. (1987). Flocks, Herds, and Schools: A Distributed Behavioral Model. *SIGGRAPH Computer Graphics*, 21(4), 25–34.

[6] Sylvester, T. (2013). *RimWorld*. Ludeon Studios.

[7] Von Neumann, J., & Morgenstern, O. (1944). *Theory of Games and Economic Behavior*. Princeton University Press.